

Machine Learning Foundations for Economic Mapping

A Guide for Economists

Andrea Salem

2026-03-03

Table of contents

1	Introduction	2
2	Statistical Learning as Posterior Updating	2
2.1	From OLS to gradient descent	2
2.2	The Adam optimizer	3
2.3	Learning rate as a practical diagnostic	3
3	The U-Net Architecture for Spatial Segmentation	4
3.1	From image classification to pixel-level prediction	4
3.2	Encoder-decoder structure	4
3.3	Residual connections and gradient flow	5
4	Regularization: Explicit Penalties vs. Implicit Bias	5
4.1	The econ toolbox: LASSO and RIDGE	5
4.2	Why explicit penalties are insufficient in deep learning	6
4.3	Implicit regularization through optimization	6
4.4	Regularization in practice: this project	7
5	Loss Function: Focal Tversky for Class Imbalance	7
5.1	The class imbalance problem	7
5.2	Segmentation metrics: IoU, mIoU, and PIoU	8
5.3	Focal Tversky loss	9
5.4	Label noise and measurement error	9
6	Foundation Models: Informative Priors for Remote Sensing	10
6.1	The Bayesian interpretation	10
6.2	Why ResNet is the wrong prior for remote sensing	11
6.3	Google Alpha Earth as the remote sensing foundation model	11
6.3.1	Representation learning as dimensionality reduction	11
6.3.2	Why AE beats ImageNet as a prior	12
6.3.3	Our approach: AE as input features, not encoder replacement	12

6.3.4	Experiment suite (5 experiments)	12
7	Stage 1: Diagnostic and Stage 1b Fix	13
7.1	What we observed in Stage 1	13
7.2	Stage 1b: targeted fixes	13
8	Limitations and Future Validation	14
9	References	15

1 Introduction

Economic measurement increasingly relies on machine learning models trained on satellite imagery. This guide introduces the core concepts needed to understand, evaluate, and extend such methods — written for researchers with an econometrics background but limited exposure to deep learning.

The organizing theme is **generalization**: how a model trained on one set of labeled examples learns representations that remain valid out-of-sample. This is the central challenge in econometrics (external validity, overfitting, regularization) and in machine learning, though the vocabulary and technical machinery differ substantially. Understanding where they converge — and where they diverge — is the goal.

Throughout, we use the concrete problem of detecting buildings from Sentinel-2 satellite imagery as a running example. The task is: given a 128×128 pixel patch of 10m-resolution multispectral imagery over Rwanda, produce a pixel-level probability map indicating which pixels contain building structures.

2 Statistical Learning as Posterior Updating

2.1 From OLS to gradient descent

Ordinary least squares minimizes a quadratic loss over a finite-dimensional parameter space. With a fixed model class and well-posed identification, the OLS estimator has a closed form: $\hat{\beta} = (X'X)^{-1}X'y$. The key feature is that the entire dataset is used in a single computation to find the exact minimizer.

Neural network training replaces this with **stochastic gradient descent (SGD)**: the loss is minimized iteratively, using small random subsets of the data (batches) to estimate the gradient at each step. For a loss function $\mathcal{L}(\theta)$ and a batch $\mathcal{B}$:

$$\theta_{t+1} = \theta_t - \alpha \cdot \nabla_{\mathcal{B}} \mathcal{L}(\theta_t)$$

where α is the **learning rate** — the step size governing how strongly the model updates its parameters at each iteration.

The Bayesian analogy is useful here. Each epoch (full pass through the training data) is analogous to a Bayesian posterior update: the model’s current parameters represent a prior belief about the prediction function, and exposure to a new batch of data produces a posterior update. The learning rate governs how strongly the prior is revised in light of new data. A very high learning rate means large revisions per step; a very low learning rate means the model converges slowly but more stably.

2.2 The Adam optimizer

Modern neural network training uses **Adam** (Adaptive Moment Estimation; Kingma & Ba 2014), which improves on vanilla SGD in two ways:

1. **Momentum** (first moment, $\beta_1 = 0.9$): the update direction is a weighted average of past gradients, smoothing out noise in the gradient estimate.
2. **Adaptive learning rates** (second moment, $\beta_2 = 0.999$): each parameter receives its own effective learning rate, scaled inversely by the square root of its recent gradient variance. Parameters with consistently large gradients receive smaller updates; parameters with small or noisy gradients receive larger updates.

The update rule for parameter θ_j at step t is:

$$\theta_{t,j} \leftarrow \theta_{t-1,j} - \alpha \cdot \frac{\hat{m}_{t,j}}{\sqrt{\hat{v}_{t,j} + \epsilon}}$$

where $\hat{m}_t$ and $\hat{v}_t$ are bias-corrected estimates of the first and second gradient moments. In practice, β_1 and β_2 are almost never tuned. The critical hyperparameter is the base learning rate α .

AdamW adds an explicit weight decay term, $\theta_t \leftarrow \lambda \theta_{t-1}$, applied at each step independently of the gradient. This is the standard variant used in modern practice and is the optimizer used in this project (weight decay $\lambda = 0.05$).

2.3 Learning rate as a practical diagnostic

Because Adam adapts per-parameter learning rates, it might seem that the base α is less important. This is not the case: α controls the maximum step size across all parameters, and setting it too high leads to **overshoot**: the optimizer takes a step that crosses the loss minimum and lands on the other side. With each step, the trajectory oscillates around but never converges to the minimum.

The diagnostic signature of overshoot is unambiguous: **both training and validation metrics degrade simultaneously**. This is distinct from overfitting, where the model memorizes training data and training metrics improve while validation metrics worsen. The two failure modes have different causes and different fixes:

Failure mode	Training metric	Validation metric	Fix
Overfitting	Improves $\rightarrow$	Degrades $\downarrow$	Regularization, more data
LR overshoot	Degrades $\downarrow$	Degrades $\downarrow$	Reduce learning rate

Failure mode	Training metric	Validation metric	Fix
--------------	-----------------	-------------------	-----

Recognizing which failure has occurred is the first step in diagnosing a training run.

The Google Deep Learning Tuning Playbook (Godbole et al. 2023) — an extensive practical guide from Google Research — formalizes this diagnostic approach. It recommends classifying hyperparameters as *scientific* (what you’re studying), *nuisance* (must be re-optimized for each experiment), or *fixed* (held constant, creating caveats). The learning rate is nearly always a nuisance parameter: it must be re-tuned whenever anything changes — architecture, batch size, loss function, or regularization. The playbook further recommends **quasi-random search** over log-uniform LR ranges rather than single-point experiments, and identifies two additional practices that interact with LR choice:

1. **Learning rate warmup**: linearly ramping LR from near-zero over the first few epochs. The playbook identifies warmup as the most reliably transferable hyperparameter for resolving early training instability — it allows the model to access higher (and potentially better) peak LRs without divergence.
2. **Fixed LR schedules over validation-sensitive schedules**: the playbook explicitly recommends against ReduceLROnPlateau-style schedules, calling them “brittle and not easily reproducible.” It recommends **cosine decay** or **linear decay** with a fixed schedule determined by `max_train_steps`, because these are deterministic, reproducible, and enable fair comparison across runs.

These recommendations have direct implications for Stage 1b (see §7).

3 The U-Net Architecture for Spatial Segmentation

3.1 From image classification to pixel-level prediction

Standard image classifiers (e.g., ResNet for ImageNet) take an entire image as input and produce a single label. Pixel-level segmentation requires a fundamentally different output: a prediction for **every** pixel in the input. The U-Net (Ronneberger, Fischer & Brox 2015) is the canonical architecture for this problem, originally developed for biomedical image segmentation with limited labeled data — a setting directly analogous to economic mapping.

3.2 Encoder-decoder structure

The U-Net consists of two components:

Encoder (contracting path): a sequence of convolutional blocks followed by downsampling operations. At each level, the spatial resolution is halved and the number of feature maps (channels) is doubled. The encoder learns *what* features are present in the image — at coarser and coarser spatial scales. In the building detection context, the encoder learns spectral and textural signatures

associated with built-up surfaces across a hierarchy of spatial scales (individual building roofs → blocks → neighborhoods).

Decoder (expanding path): the reverse process. Upsampling operations increase spatial resolution, and convolutional blocks reconstruct the full-resolution output. The decoder learns *where* features are located — which pixels have which properties.

Skip connections are the key architectural innovation: feature maps from each encoder level are concatenated with the corresponding decoder level before further processing. This provides the decoder with direct access to high-resolution spatial information that would otherwise be lost during downsampling. The skip connections are the mechanism by which the architecture achieves precise localization despite aggressive downsampling.

```

Input (128×128×3) → [Conv] → [Downsample] → [Conv] → Bottleneck
                    ↓           ↓
Output (128×128×1) ← [Conv+Skip] ← [Upsample] ← [Conv+Skip] ← [Upsample]

```

3.3 Residual connections and gradient flow

Skip connections in U-Net are related to residual connections in ResNets (He et al. 2015). The key insight: without skip connections, gradients must travel through every layer during backpropagation. In deep networks, this causes the gradient signal to shrink exponentially with depth (vanishing gradients), preventing early layers from learning. Skip connections provide a direct path from the loss to any layer in the network, ensuring that gradients remain well-conditioned throughout training.

A practical consequence for learning rate choice: because gradients reach every layer at full strength, even small changes in α produce proportionally larger effects than in architectures without skip connections. This increases LR sensitivity and partially explains why training instability manifests earlier in U-Net training than in shallower networks.

4 Regularization: Explicit Penalties vs. Implicit Bias

4.1 The econ toolbox: LASSO and RIDGE

Economists are familiar with regularization through penalized regression:

- **LASSO** (L1 penalty, Tibshirani 1996): adds $\lambda \sum_j |\beta_j|$ to the loss. Induces sparsity: many coefficients are set exactly to zero, performing automatic variable selection. Useful when the true model is sparse.
- **RIDGE** (L2 penalty, Hoerl & Kennard 1970): adds $\lambda \sum_j \beta_j^2$ to the loss. Shrinks all coefficients toward zero proportionally, without inducing sparsity. Reduces variance at the cost of bias; useful when all predictors carry some signal.

Both work by constraining the magnitude of parameters, forcing the model to “stay small” in a well-defined sense. In a fixed-dimensional linear model with a convex loss, these penalties reliably prevent overfitting by limiting model complexity.

4.2 Why explicit penalties are insufficient in deep learning

Deep neural networks are massively **overparameterized**: a small U-Net (unet_tiny) has roughly 10,000 parameters and is trained on millions of pixels. An overparameterized model can fit any dataset — including a randomly labeled one — with zero training error. In this regime, adding an L1 or L2 penalty shifts the loss minimum slightly but does not prevent the network from finding a sharp, memorizing solution.

The geometry of the loss surface in high-dimensional parameter space is fundamentally different from a linear model. Classical regularization theory, which relies on convexity and a fixed hypothesis class, does not directly transfer.

4.3 Implicit regularization through optimization

The empirical observation — and theoretical finding — is that gradient descent in overparameterized networks has an **implicit bias** toward solutions that generalize well, even without explicit regularization. Specifically, SGD and Adam tend to converge to **flat minima** of the loss surface, rather than sharp ones (Keskar et al. 2017).

A flat minimum is a solution where the loss is low in a large neighborhood of the parameter vector: small perturbations to the weights do not change predictions much. Intuitively, a model at a flat minimum has learned general features of the data-generating process rather than idiosyncratic patterns of the training sample. A sharp minimum, by contrast, corresponds to a model that memorizes specific training examples — the loss is very low at one point but rises steeply in any direction.

The learning rate plays a critical role, though the relationship is more nuanced than a simple monotonic mapping:

- **High LR**: large gradient steps inject noise into the optimization trajectory. Empirically, this tends to correlate with convergence to sharper minima, though the mechanism is debated. Keskar et al. (2017) showed that large *batch size* (which reduces gradient noise) is strongly associated with sharp minima; high LR interacts with batch size to determine the effective noise level.
- **Low LR**: smaller steps reduce oscillation and tend to converge to flatter basins — but this also depends on loss landscape geometry, which varies across architectures and loss functions.

Caveat: the flat-vs-sharp minima framework is a useful heuristic, not a proven theorem. Recent work has questioned whether sharpness alone predicts generalization (Dinh et al. 2017; Andriushchenko et al. 2023), and the interaction between optimizer, architecture, and data distribution matters. The practical takeaway remains valid: learning rate is the primary lever for controlling optimization dynamics, and tuning it carefully matters more than adding explicit penalties.

This is the sense in which Adam with suitable learning rates “achieves regularization for free”: no explicit penalty is needed; the optimizer dynamics themselves produce a bias toward generalizable

solutions. Weight decay in AdamW provides a mild additional regularizing effect analogous to RIDGE, but the primary regularization comes from the optimization process itself.

4.4 Regularization in practice: this project

Technique	What it does	Parallel in econometrics
Weight decay (AdamW, $\lambda = 0.05$)	Shrinks weights at each step	RIDGE penalty
Early stopping (patience=20 on val AUC)	Stops training before memorization sets in	Model selection by hold-out
Spatial train/val split	Val set is geographically distinct (N Rwanda vs. S/Central)	Out-of-sample evaluation
Negative downsampling (neg:pos = 1:1)	Prevents memorization of the dominant class	Balanced sampling
ReduceLROnPlateau (Stage 1b)	Reduces LR when val AUC plateaus $\rightarrow$ flatter minima	Adaptive shrinkage

The spatial train/val split deserves emphasis. Rather than holding out a random subset of pixels, the validation set consists of geographically distinct chunks (Northern Rwanda: Musanze and northeastern regions), while training covers Southern and Central Rwanda including Kigali. This is an **out-of-distribution (OOD)** evaluation: the model must generalize to a region it has not seen, with different economic geography. This is harder than a random split and more representative of the global application goal — ultimately, the model must generalize to countries it was never trained on.

5 Loss Function: Focal Tversky for Class Imbalance

5.1 The class imbalance problem

In the Rwanda validation set, building pixels constitute **0.54%** of the total — a severe class imbalance. A model that predicts “not building” everywhere achieves 99.46% accuracy. Standard metrics and loss functions are inadequate in this setting:

Metric	Problem for imbalanced data
Accuracy	99.46% by always predicting 0 — uninformative
Standard binary cross-entropy loss	Dominated by negative examples — model ignores minority class
ROC AUC	Optimistic — includes trivially easy negative examples at very low FPR

Metric	Problem for imbalanced data
PR AUC (Average Precision)	Correct — measures precision at each recall level; insensitive to class ratio

The Precision-Recall curve is the appropriate diagnostic for this problem. A random classifier achieves PR AUC equal to the positive rate (0.0054). The trained model should substantially exceed this baseline.

5.2 Segmentation metrics: IoU, mIoU, and PIoU

Beyond classification metrics like AUC and PR-AUC — which measure ranking ability at the pixel level — segmentation tasks benefit from metrics that capture **spatial overlap quality**: not just whether a pixel is correctly classified, but whether the predicted mask correctly delineates the shape and extent of the target object.

Intersection-over-Union (IoU), also called the Jaccard index, is the standard:

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|} = \frac{TP}{TP + FP + FN}$$

where A is the predicted mask and B is the ground truth. $\text{IoU} = 1$ means perfect overlap; $\text{IoU} = 0$ means no overlap. It is sensitive to both false positives and false negatives simultaneously — unlike precision (which ignores FN) or recall (which ignores FP).

mIoU (mean IoU) averages IoU across classes. For binary segmentation (building / background), this is the mean of the two class-specific IoU scores. It is the standard summary metric in semantic segmentation benchmarks (Pascal VOC, COCO, Cityscapes).

PIoU at a threshold τ is an instance-level metric:

$$\text{PIoU}_{\geq \tau} = \frac{|\{i : \text{IoU}(\hat{A}_i, A_i) \geq \tau\}|}{N_{\text{instances}}}$$

It asks: what fraction of ground truth instances (buildings, fields, etc.) are predicted with at least τ spatial overlap? A target of $\text{PIoU} \geq 0.95$ — recommended by Sebastian Sardon and used in the [Fields of the World benchmark](#) — is stringent: it requires near-exact boundary delineation, not just approximate localization.

Fields of the World (Kerner et al. 2024; github.com/fieldsoftheworld) is a multi-country Sentinel-2 agricultural field delineation benchmark. It is the closest public benchmark to this project’s setting: same imagery source (S2, 10m), same task type (pixel-level segmentation), and directly comparable evaluation methodology. Their use of mIoU and PIoU ≥ 0.95 provides a standard against which our building detection results can be contextualized, even though the segmentation targets (fields vs. buildings) differ.

Relationship to current metrics: AUC and F1 measure detection; mIoU additionally penalizes spatial imprecision in the predicted mask boundary; PIoU ≥ 0.95 is a strict geometric quality bar. For the building-density proxy goal — aggregating predictions spatially per district — mIoU is the most directly relevant: a spatially imprecise mask that over-smooths building boundaries would undercount buildings in aggregation.

Aggregation error propagation. When pixel-level predictions are aggregated to district-level economic proxies (Stage 2), classification errors do not cancel out in expectation unless they are spatially uncorrelated. Systematic errors — e.g., consistently missing small buildings in rural areas or hallucinating buildings on bright bare soil — create biased district-level estimates. This is analogous to attenuation bias in econometrics: a noisy proxy measured with systematic error biases any downstream regression. Quantifying the mapping from pixel-level IoU to district-level measurement bias is an open empirical question for this project; Stage 2 validation (predicted density vs. census counts across 30 districts) provides a first test.

5.3 Focal Tversky loss

The loss function used in this project is Focal Tversky, with $\alpha = 0.7$, $\beta = 0.3$, $\gamma = 4/3$.

Tversky index generalizes the Dice coefficient (Salehi et al. 2017):

$$T(\alpha, \beta) = \frac{TP}{TP + \alpha \cdot FN + \beta \cdot FP}$$

Setting $\alpha > \beta$ penalizes false negatives (missed buildings) more than false positives, which is appropriate when the application goal is to detect economic activity — missing a building is more costly than a false alarm. The loss is $1 - T(\alpha, \beta)$.

Focal weighting (Lin et al. 2017) down-weights easy examples and up-weights hard ones. For each pixel, the loss is multiplied by $(1 - p_t)^\gamma$, where p_t is the model’s confidence in the correct prediction. When the model is confident and correct ($p_t \approx 1$), the weight is near zero; when the model is uncertain ($p_t \approx 0.5$), the weight is high. With $\gamma = 4/3$, hard examples receive approximately $2\times$ the loss weight of easy ones.

Combined effect: the Focal Tversky loss strongly penalizes missed buildings, forces the model to focus training effort on the hardest pixels, and naturally handles class imbalance without requiring resampling. The tradeoff is training instability at high learning rates: the focal weighting creates large gradient spikes on uncertain pixels, which amplify LR overshoot. This interaction between the loss function and the learning rate is a primary contributor to the training instability observed in Stage 1.

5.4 Label noise and measurement error

The training labels — Google Open Buildings v3 polygons rasterized to 10m — are themselves model outputs, not ground truth. Open Buildings is a neural network trained on high-resolution imagery; its confidence scores reflect model uncertainty, not verified building presence. At the confidence threshold used (≈ 0.75), the labels are reliable in dense urban areas but noisier in peri-urban and rural settings where building footprints are smaller and spectrally ambiguous.

This introduces **measurement error in the dependent variable**: the model is trained to predict a noisy proxy of the true building mask. In linear regression, classical measurement error in y increases variance but does not bias coefficient estimates. In nonlinear models (neural networks),

the effect is less predictable — systematic label errors can distort learned representations. Two concrete risks:

1. **Rural undercount:** Open Buildings misses small structures (mud brick, thatch) that are spectrally similar to bare soil. The model learns to ignore these, creating a systematic gap in rural building density estimates.
2. **Urban overcount:** bright surfaces (parking lots, industrial roofs) may be labeled as buildings at lower resolution. The model inherits these false positives.

Mitigation strategies (not yet implemented):

- **Confidence-weighted loss:** weight each pixel’s loss contribution by the Open Buildings confidence score, down-weighting uncertain labels.
- **Label cleaning:** visual inspection of label quality in a sample of training patches, with manual correction of systematic errors.
- **Cross-validation against independent labels:** compare predictions against OSM building footprints or manual digitization in a test area.

6 Foundation Models: Informative Priors for Remote Sensing

6.1 The Bayesian interpretation

The connection between Bayesian inference and modern machine learning runs deeper than the epoch-as-posterior-update analogy. Consider the problem of learning with scarce labeled data — a defining feature of economic applications in data-limited settings.

In Bayesian macro, the response to scarce data is a well-specified prior. A VAR estimated on 30 years of quarterly data with 20 variables is severely underdetermined without priors on coefficient magnitudes and cross-equation correlations. The Minnesota prior, natural conjugate priors in BVAR models (see Primiceri 2005 on TVP-VARs), and shrinkage priors (Giannone, Lenza & Primiceri 2015) are all strategies for using prior information to achieve identification when the likelihood alone is insufficient.

The same logic applies in deep learning with limited labels. Training a U-Net from random initialization on 12 chunks of Rwanda imagery (roughly 3 billion pixels, 0.54% positive) works — but requires careful regularization. The alternative is to start from a model that has already learned to extract meaningful features from the imagery: a **foundation model**.

A foundation model is a neural network trained on a very large corpus of unlabeled data using self-supervised objectives. The trained encoder represents an **informative prior**: a set of learned feature representations that are useful across a wide range of downstream tasks. Fine-tuning on a labeled dataset then performs the posterior update, specializing the representations to the specific task. The analogy to Bayesian inference is precise:

Bayesian inference	Foundation model + fine-tuning
Prior distribution	Foundation model weights
Likelihood (labeled data)	Fine-tuning dataset + task-specific loss
Posterior distribution	Fine-tuned model weights
More data $\rightarrow$ likelihood dominates prior	More labels $\rightarrow$ fine-tuning dominates pretrained weights

The practical implication: with an informative prior (strong pretrained encoder), fewer labeled examples are needed to achieve good task performance.

6.2 Why ResNet is the wrong prior for remote sensing

ResNet (He et al. 2015) is a deep convolutional network pretrained on **ImageNet**: 1.2 million natural photographs of everyday objects (animals, furniture, vehicles, etc.). The features learned by a ResNet encoder reflect the statistical structure of photographs: perspective distortion, shadows, object-centered composition, RGB color statistics.

Sentinel-2 satellite imagery is structurally different: a top-down perspective, 10–13 spectral bands including near-infrared and short-wave infrared (invisible to human cameras), no perspective effects, and spatial statistics that reflect land cover rather than objects. Using a ResNet encoder as a prior for satellite imagery means starting from representations that are largely irrelevant to the downstream task — equivalent to specifying a prior concentrated on the wrong region of parameter space.

6.3 Google Alpha Earth as the remote sensing foundation model

Alpha Earth (Google DeepMind) is a transformer-based geospatial foundation model pretrained on hundreds of millions of Sentinel-2 patches globally. Available on GEE as `GOOGLE/SATELLITE_EMBEDDING/V1/ANNUAL`: 64 bands (A00–A63), 10m resolution, annual composites from 2017–2025, CC-BY 4.0 license.

The pretraining objective is **masked spectral band prediction**: given a subset of spectral bands for a patch, predict the held-out bands. This forces the model to learn the spatial and spectral covariance structure of satellite imagery — exactly the structure that is predictive of land cover, built-up density, and economic activity.

6.3.1 Representation learning as dimensionality reduction

The connection to representation learning is direct. Just as factor analysis in psychology compresses hundreds of personality questionnaire items into 5 latent factors (Big 5 / OCEAN), Alpha Earth compresses all geospatial information observable from satellite imagery into 64 learned “traits” per pixel. Each embedding dimension captures a different aspect of the geospatial signal — land use type, vegetation density, urban morphology, infrastructure patterns — learned entirely from the statistical structure of satellite data. This is a core idea in representation learning (cf. ICLR — the International Conference on *Learning Representations*).

6.3.2 Why AE beats ImageNet as a prior

Sirko et al. (2021) used ResNet-50 pretrained on ImageNet (1.2M photographs of everyday objects) as a feature extractor for building detection. ImageNet features encode the statistical structure of photographs: perspective distortion, RGB color statistics, object-centered composition. These features are largely irrelevant for satellite imagery. Alpha Earth, pretrained on actual satellite data, provides domain-specific representations that directly encode the spatial and spectral patterns relevant to remote sensing tasks.

6.3.3 Our approach: AE as input features, not encoder replacement

Rather than replacing the U-Net encoder with a pretrained AE encoder (the standard fine-tuning approach), we treat AE’s 64-band output as **additional input features** alongside raw S2 spectral bands. This preserves the simplicity of our existing pipeline:

- **Two-branch early fusion:** S2 bands (raw spectral reflectance, 12 bands) and AE bands (transformer-learned embeddings, 64 bands) have fundamentally different statistical distributions. Separate initial convolutional blocks let the model learn modality-specific feature extractors before merging into a shared U-Net encoder-decoder.
- **Data pipeline unchanged:** Both modalities are stacked into a single 76-band GeoTIFF per chunk. The model’s `s2_channels` config parameter tells it where to split the input.
- **No frozen encoder complexity:** All parameters are trainable from the start, using the same conservative LR (5e-6) validated in Stage 1b.

This approach is motivated by the hypothesis that AE embeddings encode richer economic signals (land use, infrastructure type, commercial activity patterns) that raw spectral bands alone cannot capture — potentially breaking through the $r \sim 0.4$ district-level correlation barrier.

6.3.4 Experiment suite (5 experiments)

#	Input	Bands	Architecture	Purpose
0	S2 NIR	3	unet_tiny_1b	Baseline (AUC=0.976)
1	S2 NIR+NDVI	4	unet_tiny_1b	NDVI separates FP/TP
2	S2 all	10	unet_medium	S2 saturation check
3	AE only	64	unet_medium	AE standalone quality
4	S2+AE	76	unet_dual_branch	Full two-branch fusion

Primary evaluation metric: district-level Pearson r with census, not just pixel AUC.

7 Stage 1: Diagnostic and Stage 1b Fix

7.1 What we observed in Stage 1

Stage 1 training (buildingsNIR-Rwanda2022) ran for 26 epochs (9h 49m on CPU). The best checkpoint was saved at epoch 6 (val AUC = 0.9301). After epoch 6, both training and validation AUC declined together before partially recovering. Training was stopped by early stopping at epoch 26 with patience of 20 epochs.

Diagnosis: the simultaneous decline in both training and validation metrics rules out overfitting (which would show diverging train/val curves). The pattern is consistent with **learning rate overshoot**: the optimizer, at $\alpha = 5 \times 10^{-5}$, was taking steps large enough to cross the loss minimum on each iteration and overshoot the other side.

Contributing factors: 1. Although the base LR of 5×10^{-5} is $20\times$ below Adam’s commonly cited default of 10^{-3} , the effective step size depends on the loss landscape, not just the nominal LR. The combination of focal Tversky gradients and U-Net skip connections creates a steeper effective landscape, making 5×10^{-5} too aggressive for this specific setting. 2. Focal Tversky loss creates large gradient spikes on uncertain pixels (the hard cases that receive $2\times$ weighting). These spikes, combined with Adam’s momentum, produce large effective updates that overshoot the loss minimum. 3. U-Net skip connections deliver gradients at full strength to all layers simultaneously, amplifying the overshoot across the entire network.

7.2 Stage 1b: targeted fixes

Two changes were made for the Stage 1b run:

Fix 1: Reduce base learning rate $10\times \alpha : 5 \times 10^{-5} \rightarrow 5 \times 10^{-6}$

This directly reduces the step size, preventing the optimizer from overshooting the loss minimum. The $10\times$ reduction is the standard first intervention when both train and val metrics simultaneously degrade.

Fix 2: ReduceLROnPlateau callback

Added to the training loop: if `val_auc` does not improve by at least 10^{-3} for 5 consecutive epochs, the learning rate is multiplied by 0.5. The minimum LR floor is 10^{-7} .

This implements an adaptive LR annealing schedule: the LR starts at 5×10^{-6} and automatically reduces as training converges. The Bayesian analogy: early epochs use a broad prior (high LR = large updates), and later epochs narrow the posterior (low LR = fine-grained convergence). No prior specification of total training length is required — the schedule is driven by the empirical signal from the validation set.

Important caveat on ReduceLROnPlateau. The Google Deep Learning Tuning Playbook (Godbole et al. 2023) explicitly recommends *against* validation-sensitive schedules like ReduceLROnPlateau, calling them “brittle and not easily reproducible.” The concern is that the schedule becomes path-dependent: two runs with the same hyperparameters can produce different LR trajectories due to stochastic differences in validation metrics, making it impossible to compare experiments fairly. The playbook recommends **cosine decay** — a deterministic schedule that decays from the initial LR to near-zero over a fixed number of steps — combined with **linear warmup** over the

first few epochs. This is a planned improvement for Stage 1c: replace ReduceLROnPlateau with cosine decay + 5-epoch warmup, which may also permit recovering the higher base LR of 5×10^{-5} with warmup providing the stability that simply reducing the LR was meant to achieve.

	Stage 1	Stage 1b	Stage 1c (planned)
Base LR	5×10^{-5}	5×10^{-6}	5×10^{-5} (with warmup)
LR schedule	Fixed	ReduceLROnPlateau	Cosine decay
Warmup	None	None	Linear, 5 epochs
Val AUC trajectory	Peak ep 6 $\rightarrow$ degrades	Expected: monotonic	Expected: monotonic
Architecture	unet_tiny	unet_tiny_1b	unet_tiny_1c

All other settings — architecture depth, bands, loss function, training/validation split — are held constant. The comparison between Stage 1, 1b, and 1c isolates the effects of LR magnitude vs. schedule design.

8 Limitations and Future Validation

This guide is a conceptual and diagnostic framework, not an empirical benchmark. Several claims require empirical validation that is planned but not yet completed:

Empirical gaps:

1. **Stage 1b results not yet presented.** The LR overshoot diagnosis and proposed fix ($10\times$ LR reduction + ReduceLROnPlateau) are hypothesized to produce monotonic convergence. Quantitative results — whether val AUC improves monotonically, final metric values, and comparison to Stage 1 — will be added when Stage 1b training completes.
2. **No ablation studies.** The individual contributions of focal Tversky loss (vs. Dice, weighted BCE), architecture (unet_tiny vs. deeper variants), and weight decay have not been isolated. Each is justified on theoretical grounds, but their empirical marginal effects are unknown.
3. **Foundation model comparison is aspirational.** The Alpha Earth discussion is forward-looking. No quantitative comparison between random initialization, ImageNet pretraining, and Alpha Earth fine-tuning has been conducted. This comparison is planned for Stage 3.
4. **Single training run.** Stage 1 results are from a single run with a single random seed. Variance across seeds, and the sensitivity of the best-epoch metric to initialization, have not been assessed.

Methodological gaps:

5. **No uncertainty quantification.** Reported metrics (AUC, F1, mIoU) are point estimates with no confidence intervals, bootstrap estimates, or cross-run variability. For econometric applications, uncertainty bounds on predicted building density are essential — particularly when aggregating to district-level proxies.

6. **Overshoot diagnosis is inferential.** The attribution of Stage 1 degradation to LR overshoot is based on metric trajectory analysis (simultaneous train+val decline). Direct evidence — gradient norm tracking, loss landscape visualization, or sharpness-aware perturbation tests — would strengthen the diagnosis but has not been collected.
7. **Geographic generalization untested beyond Rwanda.** The OOD validation (North vs. South Rwanda) tests spatial generalization within one country. Cross-country generalization — the ultimate goal — has not been evaluated. The extent to which building spectral signatures transfer across climatic zones, urban morphologies, and construction materials is an open question.

These gaps define the empirical agenda for subsequent stages. This document will be updated as results become available.

9 References

- Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv:1412.6980*.
- Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. *MICCAI 2015*.
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Deep residual learning for image recognition. *arXiv:1512.03385*.
- Vaswani, A., et al. (2017). Attention is all you need. *NeurIPS 2017*.
- Keskar, N. S., Mudigere, D., Nocedal, J., Smelyanskiy, M., & Tang, P. T. P. (2017). On large-batch training for deep learning: Generalization gap and sharp minima. *ICLR 2017*.
- Brown, C. F., et al. (2022). Dynamic World, near real-time global 10m land use land cover mapping. *Scientific Data*.
- Primiceri, G. E. (2005). Time varying structural vector autoregressions and monetary policy. *Review of Economic Studies*, 72(3), 821–852.
- Salehi, S. S. M., Erdogmus, D., & Gholipour, A. (2017). Tversky loss function for image segmentation using 3D fully convolutional deep networks. *Machine Learning in Medical Imaging*.
- Lin, T. Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal loss for dense object detection. *ICCV 2017*.
- Dinh, L., Pascanu, R., Bengio, S., & Bengio, Y. (2017). Sharp minima can generalize for deep nets. *ICML 2017*.
- Andriushchenko, M., Flammarion, N., & Hein, M. (2023). Why do we need weight decay in modern deep learning? *arXiv:2310.04415*.
- Giannone, D., Lenza, M., & Primiceri, G. E. (2015). Prior selection for vector autoregressions. *Review of Economics and Statistics*, 97(2), 436–451.
- Godbole, V., Dahl, G. E., Gilmer, J., Shallue, C. J., & Nado, Z. (2023). Deep learning tuning playbook. *GitHub, Google Research*. github.com/google-research/tuning_playbook
- Kerner, H., et al. (2024). Fields of the World: A machine learning benchmark dataset for global agricultural field boundary segmentation. *arXiv:2409.16252*. github.com/fieldsoftheworld